

תרגיל בית 3

אופן ההגשה:

- במידה ובתרגיל מופיעה דוגמא של פלט מסוים, הקפידו שהפלט שלכם יהיה זהה
- אם בתרגיל הוזכרו במפורש שמות מחלקות/משתנים וכד', או קבצים בהם יש להשתמש – יש לדייק בשמות, ולהשתמש בקבצים שהוגדו. יש לממש את השאלות ב java ולהגיש רק קבצי java ללא קבצים אחרים של הפרוייקט.
- **יש לרשום את שם פרטי, שם משפחה ות"ז המגישים בראש כל קובץ המהווה חלק מפיתרון התרגיל. בנוסף יש להגיש קובץ word המכיל את השמות שלכם בעברית ותשובות על השאלות בכתב.**
- במידה והמטלה מכילה יותר מקובץ אחד, יש לכווץ את כלל הקבצים לקובץ zip (ולא פורמטים אחרים כמו rar), כאשר שם הקובץ המכווץ כולל את ת"ז המגישים. למשל, אם הגישו את העבודה בעלי ת"ז 11111 ו-22222, שם הקובץ יהיה 11111_22222.zip. לאחר הכיווץ, העלו את קובץ ה zip בלבד ללימוד.
- **בנוסף, בקובץ word ובקבצים עם ה-main של כל שאלה יש לרשום קישור ל-GitHub עם הקוד שלכם שפיתחתם אותו ב-Git.**
- ההגשה בזוגות בלבד (אלא אם כן אושר אחרת ע"י המרצה)
- יש להגיש את הפתרון עד למועד שנקבע בלימוד
- חלק מהנבדק בתרגיל הוא עמידה בנקודות הנ"ל. הקפידו לעמוד בהן.
- אישורי הארכה יינתנו ע"י המרצה בלבד.
- את כל השאלות יש לשאול בפורום לעבודה 3.

בהצלחה!

שאלה 1

התבוננו בקוד הבא:

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class Account {
    private double balance;
    private int id;
    private Lock lock = new ReentrantLock();

    public Account(int id, double balance) {
        this.id = id;
        this.balance = balance;
    }

    private boolean withdraw(double amount) {
        if (lock.tryLock()) {
            // Some work like accessing the database
            balance = balance - amount;
            return true;
        }
        return false;
    }

    private boolean deposit(double amount) {
        if (lock.tryLock()) {
            // Some work like accessing the database
            balance = balance + amount;
            return true;
        }
        return false;
    }

    public boolean transfer(Account dest, double amount) {
        if (withdraw(amount))
            if (dest.deposit(amount))
                return true;
            else
                deposit(amount);
        return false;
    }
}
```

```

public class Transaction implements Runnable{
    private Account a1, a2;
    private double amount;

    public Transaction(Account a1, Account a2, double amount) {
        this.a1 = a1;
        this.a2 = a2;
        this.amount = amount;
    }

    @Override
    public void run() {
        while(!a1.transfer(a2, amount));
        System.out.println(Thread.currentThread().getName()+
            " is finished!");
    }
}

public class Main {

    public static void main(String[] args) {
        Account a1 = new Account(1, 1000);
        Account a2 = new Account(2, 1000);

        Thread t1 = new Thread(new Transaction(a1,a2,20),"trans-1");
        Thread t2 = new Thread(new Transaction(a2,a1,20),"trans-2");

        t1.start();
        t2.start();
    }
}

```

אילו בעיות בטיחות עלולות להתעורר בקוד המדובר? לכל סוג בעיה, הסבירו את הבעיה במילים ותנו דוגמה מוחשית של רצף פעולות שיביא לבעיה שתיארתם.

הערה: הפקודה lock.tryLock() דומה ל- synchronized(this) {...} . היא תנסה לנעול את המנעול של המופע this, ותחזיר אמת אם התהליכון הצליח להשיג נעילה.

יותר על ReentrantLock ועל tryLock() אפשר לקרוא [כאן](#).

שאלה 2

פתחו חבילה בשם `assig3_2` וממשו תוכנית המדמה משחק עץ או פלי, עם החוקים הבאים:

1. במשחק ישנם שני משתתפים, ומטבע אחד. לכל משתתף במשחק ישנו מונה, הסופר את כמות הפעמים שהתקבל עץ (מתקבל הערך `true` מהפונקציה `flipCoin()`).
 2. המשחק נגמר, לאחר ששני המשתתפים הטילו את המטבע 10 פעמים.
 3. המנצח הוא השחקן עם מספר ההטלות הרב ביותר שבהן התקבל עץ.
- המחלקה `GamePlay` מייצגת את המשחק עצמו, את המטבע ואת מספר ההטלות

ממשו את המחלקה `GamePlay` באופן הבא:

1. שדה `coin_available_` המתאר האם המטבע זמין.
 2. שדה `rounds_counter_` המתאר את כמות ההטלות שבוצעו במשחק.
 3. פונקציה `makeCoinAvail(boolean val)`:
 - a. הופכת את המטבע לזמין או לא זמין לפי הערך `val`.
 - b. במידה והמטבע הופך זמין, מודיעה על כך לשאר התהליכונים שממתינים למטבע.
 - c. מעדכנת את השדה `coin_available_` בהתאם לערך שקיבלה.
 4. פונקציה `flipCoin()`:
 - a. אם המטבע אינו זמין, התהליכון:
 - i. ימתין עד שהמטבע יהפוך זמין (זכרו להשתמש ב `guarded suspension`)
 - ii. ידפיס את שמו (באמצעות `getName()` של המחלקה `Thread`), ואת העובדה שהוא נכנס להמתנה. למשל:
Thread-0 is waiting for coin
 - b. אם המטבע זמין, התהליכון:
 - i. ידפיס שהוא מטיל מטבע. למשל:
Thread-1 is flipping coin
 - ii. יהפוך את המטבע ללא זמין במהלך ההטלה
 - iii. יקדם את מספר ההטלות ב-1
 - iv. יגריל ראנדומאלי 0 או 1
 - v. יהפוך את המטבע שוב לזמין, ויודיע על כך לתהליכונים אחרים
- c. הפונקציה תחזיר את תוצאת ההטלה (`0` = שקר/כשלון, `1` = אמת/הצלחה)

5. פונקציה `getNumOfRounds()` :

a. מחזירה את מספר ההטלות במשחק

המחלקה `Gamer` מייצגת תהליכון שחקן אשר מטיל את המטבע שוב ושוב

ממשו את המחלקה `Gamer` באופן הבא:

1. שדה `goodFlipsCounter` הסופר את כמות ההטלות שהשחקן ביצע והצליחו

2. שדה מסוג `GamePlay` המאותחל בבנאי (האובייקט מתקבל מבחוץ)

3. פונקציית `play()` המתבצעת בלולאה, ובכל איטרציה:

a. כל עוד שלא ביצעו לתהליכון `INTERRUPT` וגם מספר ההטלות במשחק כולו קטן או שווה

ל 10:

i. נסה להטיל מטבע, ואם הצלחת קדם את `goodFlipsCounter` ב-1

ii. לך לישון למשך שנייה אחת

4. `GETTER` בשם `getScore` המחזיר את מספר ההטלות שהצליחו

המחלקה `Judge` מייצגת שופט במשחק, ההופך את המאשר לשחקנים מתי הם רשאים להטיל את

המטבע ומתי לא, ע"י הפיכת המטבע לזמין / לא זמין

ממשו את המחלקה `Judge` באופן הבא:

1. לולאה, בה כל עוד לא ביצעו לתהליכון `interrupt`:

a. השופט הופך את המטבע ללא זמין למשך שנייה

b. השופט הופך את המטבע לזמין למשך חצי שנייה

במחלקה הראשית של התוכנית:

1. צרו משחק חדש

2. צרו שני שחקנים

3. לאחר שהשחקנים מסיימים לשחק, מודפסת הודעה על השחקן שניצח (השחקן בעל הניקוד

הגבוה ביותר). למשל:

player 1 wins

במידה ולשחקנים ניקוד זהה, יודפס תיקו כך:

tie

שאלה 3

פתח חבילה בשם `assig3_3`, והכנס לתוכה את קבצי הקוד שתחת התיקיה `assig3_3` שצורפה למטלה. בשאלה זו עליכם לממש תוכנה עבור מכונה אוטומטית להכנת סלט. המכונה מורכבת מתא בעל ראש קוצץ, שאליו מוזנים באופן אוטומטי בצל, מלפפונים ועגבניות. ברגע שלתא הוזנו בצל אחד, 5 מלפפונים ו-3 עגבניות, הלהבים מכינים מהם סלט אחד. בתחילת התוכנית המשתמש בוחר כמות סלטים להכנה, והמכונה תכין את הסלטים אוטומטית תחת האילוצים הבאים:

- תהליך הזנת מלפפונים לתא הקוצץ יכול לקרות רק אם יש בתא פחות מ 5 מלפפונים. בכל מקרה אחר הזנת המלפפונים צריכה להיעצר עד שיתפנה מקום בתא הקוצץ.
- תהליך הזנת עגבניות לתא הקוצץ יכול לקרות רק אם יש בתא פחות מ 3 עגבניות. בכל מקרה אחר הזנת העגבניות צריכה להיעצר עד שיתפנה מקום בתא הקוצץ.
- תהליך הזנת בצל לתא הקוצץ יכול לקרות רק אם אין בתא בצל. בכל מקרה אחר הזנת הבצל צריכה להיעצר עד שיתפנה מקום בתא הקוצץ.
- הקוצץ יכין סלט רק כאשר בתא יש בצל אחד, 5 מלפפונים ו-3 עגבניות. בכל מקרה אחר הקוצץ ייעצר.
- כאשר הקוצץ הכין N סלטים (לפי בקשת המשתמש), הוא יעצר, ואחריו יעצרו גם התהליכונים שמזינים את הקוצץ בירקות, והתוכנית תסיים כאשר כל התהליכונים נסגרו.

לשימושכם בקוד שקיבלתם מחלקות שאותן יש לשנות ולהשלים:

- מחלקה בשם `CucumberThread` האחראית על הזנת מלפפונים לקוצץ.
- מחלקה בשם `OnionThread` האחראית על הזנת בצל לקוצץ.
- מחלקה בשם `TomatoesThread` האחראית על הזנת עגבניות לקוצץ.
- מחלקה בשם `SlicerThread` האחראית על קיצוץ הירקות.
- מחלקה בשם `SlicerMachine` המייצגת את הפונקציות של המכונה:
 - `addOneCucumber` – הזנת מלפפון אחד לתא הקוצץ
 - `addOneTomato` – הזנת עגבניה אחת לתא הקוצץ
 - `addOneOnion` – הזנת עגבניה אחת לתא הקוצץ
 - `sliceVegetables` – קיצוץ הירקות
 - `getNumOfPreparedSalads` – מחזירה את מספר הסלטים שהמכונה הכינה

עליכם לממש את המחלקות של התהליכונים כך שישתמשו במחלקה SlicerMachine להשגת המטרה שהוגדרה.

המחלקה SlicerMachine מכילה קוד ראשוני בלבד שיעבוד בהרצה **סדרתית**, שנו אותה כך שתתאים להרצה מקבילית שתקיים את הלוגיקה שהוגדרה תוך **שמירה על בטיחות להרצה מקבילית**.
דוגמא לפלט תקין אפשרי של התוכנית (שימו לב שהפלט מעיד שהוכנו 3 סלטים כמו שהמשתמש בחר, ובנוסף כל הכנת סלט קרתה רק אחרי שהיו מספיק ירקות בקוצץ) :

Please Type How Many Salads To Prepare:

3

Preparing 3 Salads...

adding one tomato to the machine

adding one cucumber to the machine

adding one tomato to the machine

adding one cucumber to the machine

adding one cucumber to the machine

adding one tomato to the machine

adding one cucumber to the machine

adding one cucumber to the machine

adding one onion to the machine

== preparing one more salad ==

adding one cucumber to the machine

adding one tomato to the machine

adding one cucumber to the machine

adding one tomato to the machine

adding one cucumber to the machine

adding one onion to the machine

adding one cucumber to the machine

adding one cucumber to the machine

adding one tomato to the machine

== preparing one more salad ==

adding one tomato to the machine
adding one cucumber to the machine
adding one tomato to the machine
adding one tomato to the machine
adding one cucumber to the machine
adding one cucumber to the machine
adding one cucumber to the machine
adding one onion to the machine
adding one cucumber to the machine
== preparing one more salad ==
adding one tomato to the machine
adding one cucumber to the machine
adding one tomato to the machine
Done

שאלה 4

יש לנו 4 תהליכונים t1, t2, t3, ו-t4 כל תהליכון מבצע בלולאה נצחית קטע קוד. כלומר קוד של 4 תהליכונים נראה כך:

```
t1:  
void run(){  
    while (true){  
        A  
    }  
}
```

```
t2:  
void run(){  
    while (true){  
        B  
    }  
}
```



```

t3:
void run(){
    while (true){
        C
    }
}

t4:
void run(){
    while (true){
        D
    }
}

public static void main(String[] args){
    Thread t1 = new Thread();
    Thread t2 = new Thread();
    Thread t3 = new Thread();
    Thread t4 = new Thread();
    t1.start();
    t2.start();
    t3.start();
    t4.start();
}

```

כאשר A, B, C, D זה בלוק של שורות קוד שהם מבצעים.

עליכם לסנכרן את התהליכונים (בעזרת אחת השיטות שלמדתם) כך שיתבצעו באופן הבא: קודם תהליכון t1 יבצע את קטע קוד A מתחילתו ועד סופו 3 פעמים, לאחר מכן תהליכון t2 יבצע את קטע קוד B מספר כלשהו של פעמים, מתחילתו ועד סופו, אחרי זה תהליכון t3 יבצע את קטע קוד C פעמיים ולאחר מכן t4 יבצע את הקטע קוד D מתחילתו ועד סופו מספר כלשהו של פעמים. אחרי הסיבוב הזה הכל מתחיל מחדש החל תהליכון t2, כלומר יתבצע קטע קוד B מספר כלשהו של פעמים, אחריו C פעמיים ואחריו D מספר כלשהו של פעמים. הסבב של $B \cdot C^2 \cdot D$ יתבצע אינסוף פעמים אחרי A^3 . הערה: מספר פעמים שקטע קוד B ו-D מתבצעים ברצף אינו נתון מראש, הדבר תלוי בתזמון התהליכונים בסוף ביצוע קטע קוד קודם.

דוגמה לתזמון אפשרי:

AAABBBBBBCCDDDDBBBCCDDDDDDBBBBBCCDBCCDD...